

Vemaque: Approximately Verifiable Remote Computation of k-Clique and Maximum Clique Problems

R.G.L.M Samarawickrama¹, D. N. Ranasinghe², T. Sritharan³

University of Colombo School of Computing, Sri Lanka

¹lmsamarawickrama@gmail.com, ²dnr@ucsc.cmb.ac.lk, ³rts@ucsc.cmb.ac.lk

Abstract— When a client needs to delegate the computation of a task to another party, there will often be a need to verify the result returned from that party, without re-executing whole of the computation. Recent work on verifiable remote computation has made a significant advancement on this problem by solving it to near practicality. However such works can only handle a restricted class of computation such as linear computations. Towards solving this gap, this paper presents *Vemaque*, a system to approximately verify two NP-complete computations (the k-clique problem and the maximum clique problem) that are not addressed in prior works. Experimental results show that *Vemaque* has achieved more than 98% accuracy of verification and reasonable performance using probabilistic proof systems.

Keywords—Verifiable remote computation, k-clique problem, Maximum Clique problem, Interactive Proofs, Probabilistically checkable proofs

I. INTRODUCTION

Recently, devices with relatively limited computational power and memory capacity (such as phones, PCs and etc.) have tended to outsource their computations and data storage to third-parties. Although the advantages of third-party computation is significant, the client can't generally trust the result returned from it since many things can go wrong in third-party computation. For instance, in cloud computing, there can be misconfigurations, malicious attacks, correlated manufacturing defects and more [3]. Therefore, in the past few years, there has been an increasing interest in verifying the correctness of the results returned from third-party computations [1].

Replicated computation used in the famous project SETI@home [2] is a possible solution to this problem, but this works only if replica faults are uncorrelated [1]. Client can audit the output, but this technique fails if a problem happens outside the selected sample [14]. Trusted computing is also another solution but it highly depends on the reliability of hardware. None of these solutions are equally available for all types of tasks. So the need for an unconditional verification of third-party computation has emerged.

What if the third-party returns its result along with a certificate that guarantees the correct execution and what if that certificate was inexpensive to check, compared to the cost of re-executing the computation in client device [1]. We call this approach Proof-based Verifiable Computation (PVC). Theoretically, researchers have been able to achieve unconditional verification of third party computations using

PVC since 1990s but lacks bringing this phenomenon to near practicality.

We can identify a number of built systems [7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17,18] which have aimed at realizing PVC using complexity theory and cryptography. However these systems can only handle linear computations [1]. Linear computations are ones with branching and cycling instructions depending only on the size of initial data whose loop bounds are known at compile time [19]. Many well-known procedures for algebraic calculations can be described as linear, including multiplications of polynomials, matrices [19] etc.

Such systems present a general approach to handle computations by either representing the computation as an equivalent set of constraints or Boolean operations. Since, linear computations can be directly represented as set of multiplications and additions, they can be compiled into a set of mathematical constraints or Boolean operations. So prior works can handle the class of linear computations. But NP-Complete (NPC) problems do not fall into this category. Therefore, existing systems cannot handle such computations.

This paper presents *Vemaque*, a built system to approximately verify two NP-complete computations: k-clique problem (KCLQ) and maximum clique problem (MCLQ) by combining the results of NP-completeness with mechanism of probabilistic proof systems (PPSs) and prior works based on PVC.

Contribution of this research is two-fold:

1. We describe how *Vemaque* is built by incorporating results of NP-completeness with similar frameworks that appear in prior works (Section II, Section III and Section IV).
2. Extensive simulations presented to evaluate the accuracy and set up cost of *Vemaque* (Section V). Results show that *Vemaque* achieves more than 98% of accuracy for the verification of both KCLQ and MCLQ with a reasonable set up cost for graph sizes up to 10,000 nodes with 2,000,000 edges.

II. RELATED WORK

A. Probabilistic Proof Systems (PPSs)

PPSs consist of two parties: Verifier (V) and Prover (P). With probabilistic proof systems, P can efficiently convince the V of a mathematical assertion. We

concentrate on two classes of PPSs in our study. They are Interactive proofs (IPs) and Probabilistically Checkable Proofs (PCPs) [1,5].

1) *Interactive Proofs (IPs)* For a language L , IP is a two-party dialog between a V and P that interact on a common input in a way satisfying the Completeness and Soundness and properties where V 's strategy is probabilistic in polynomial time.

2) *Probabilistically Checkable Proofs (PCPs)* : With PCPs, V checks only some randomly selected parts of P 's certificate/proof. V determines the correctness of that certificate only with that checked part. This highly amortizes the verifier's cost of computation.

B. Works in Verifiable Remote Computation

Goldwasser et al. [10] presents an interactive proof system (Muggles) that requires only polynomial work from verifier to verify an arbitrary computation. Muggles represents computations as certain kinds of Boolean circuits since these Boolean circuits work naturally with PPSs. Muggles uses the power of interaction (Interactive proofs) in verification process that proceeds in rounds. Muggles can handle computations that can be directly represented as a set of multiplications and additions. Muggles can't handle verifying any NP-Complete computation. *Vemaque* borrows the power of interaction for its iterative verification process from Muggles and reduces this gap.

Pepper [17], Ginger [18], Zaatar [8] and Pinocchio [7] is a line of work that represent the computation as a set of arithmetic constraints. A correct solution to these constraints corresponds to a valid transcript of the computation. This approach is more efficient and concise than the approaches with Boolean circuits [10, 11, 12, 13]. This approach too is not suitable for some real-world computations since it requires loop bounds of the computation to be known at compile time. Thus, [7, 8, 17, 18] can handle only linear computations.

Vemaque borrows the underlying computational model, representing the computation as a set of arithmetic constraints and the common verification process used in Zaatar and Pinocchio to achieve a reasonable verification for KCLQ and MCLQ that are not handled in prior work.

III. VEMAQUE'S BASE: ZAATAR AND PINOCCHIO

As we have mentioned in previous section, Zaatar [8] and Pinocchio [7] are the underlying frameworks for *Vemaque*. We present Zaatar and Pinocchio and their underlying principles for *Vemaque* in a unified framework.

Zaatar and Pinocchio are based on a strong assumption on P . That is, P has a computational bound such that P cannot solve NP hard tasks quickly and accurately in all cases. Other than this assumption, they don't assume anything about P . So, this protocol guarantees the verification even if P doesn't follow the protocol (malicious or malfunctioned). But this assumes that V is honest.

A. Zaatar and Pinocchio in brief

The protocol in Zaatar and Pinocchio can be described by four steps.

1) *Both P and V represent a computation F as a set of arithmetic constraints C* : P can satisfy these constraints if it executes F correctly on input X and satisfying these constraints with a wrong execution is very hard. Consider the following computation that compare two given variables $Z1$ and $Z2$ with IF and ELSE statement

IF ($Z1 \neq Z2$) then $Z3 = 1$, ELSE $Z3 = 0$;

Representing the above computation as set of arithmetic constraints is as follows.

$$M \cdot (Z1 - Z2) - Z3 = 0 \text{ for some value } M \\ (1 - Z3) \cdot (Z1 - Z2) = 0$$

M equals the multiplicative inverse of $(Z1 - Z2)$ if $Z1 \neq Z2$.

2) *P computes F on X* : P executes F for the input X and obtain the output Y that is supposed to be $Y = F(X)$.

3) *P satisfies C with X and Y* : After the computation, P satisfies the set of constraints C with X and Y . These set of satisfied constraints works as the certificate. Then, P encodes this certificate into a long string π and sends it to V .

4) *V checks π probabilistically.* : V receives Y along with π . Then, V checks the validity of π approximately by inspecting a small number of randomly-chosen locations to accept or reject Y .

B. Vemaque over Zaatar and Pinocchio

Since Zaatar and Pinocchio can handle only linear computations, handling NP-complete computations is not possible with them. Reducing this gap, *Vemaque* goes a step further from Zaatar and Pinocchio by presenting a verifiable computation mechanism for two NP-complete computations.

Zaatar and Pinocchio represents the computation as set of equivalent arithmetic constraints based on necessary conditions where satisfying these constraints is equivalent to solving problem. *Vemaque* represents the computation as set of arithmetic constraints based on sufficient conditions where satisfying these constraints doesn't solve the problem. But *Vemaque* offers a reasonable set up cost for both V and P with a higher accuracy.

Vemaque is fine tuned for only two computations. But Zaatar and Pinocchio can handle a restricted class of computations with reasonable cost. So among number of works in PCP, Zaatar and Pinocchio have a significant value and importance. But client's cost and prover's overhead is still higher in Zaatar and Pinocchio.

IV. MECHANISM

Vemaque handles verification for KCLQ and MCLQ and it happens outside the client machine.

The objective of KCLQ is to find k number of vertices $V^C \subseteq V$, any two of which are adjacent and MCLQ is to find the largest number of vertices $V^C \subseteq V$, any two of which are adjacent for an undirected graph $G = (V, E)$. In the clique decision problem (CLIQUE), the input is G and a number k , and the output is a Boolean value: true if the graph contains a k -clique, and false otherwise. [24] has proven that CLIQUE is NP-complete. Since decision is no harder than

an optimization problem, both KCLQ and MCLQ are also NP-complete.

A. Unified Framework

The operation of *Vemaque* can be described by seven steps as depicted in Figure 1.

1) Verifier represents the computation F (either KCLQ or MCLQ) as arithmetic constraint C . In *Vemaque*, C' and C'' are arithmetic constraints for KCLQ and MCLQ respectively. Then, client sends the input X to the server. X is the input graph G for MCLQ and G with positive integer k for KCLQ

2) Prover also represents F as arithmetic constraint C . Then P executes F on X and sends output V^C to client. Bron and Kerbosch algorithm [23] guarantees to generate all maximal cliques. Since maximum clique is necessarily to be a maximal clique (inverse is not true), finding largest maximal clique corresponds to finding a maximum clique. We have used this method to solve MCLQ in our prover implementation since it guarantees to find the maximum clique [25].

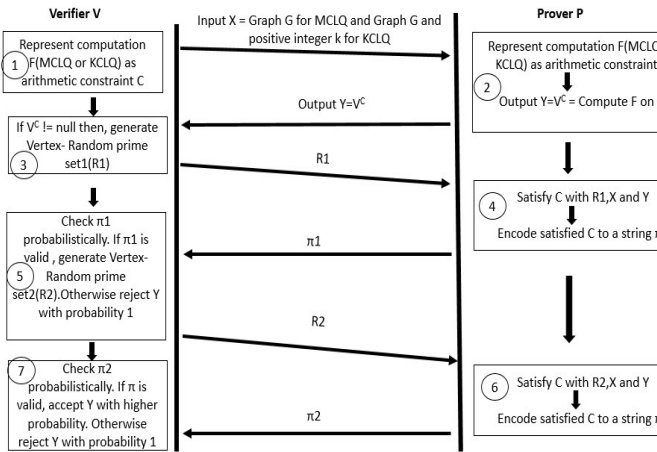


Figure 1 : Framework of Vemaque

3) V first checks whether V^C is empty or not. If V^C is not empty, V starts the interactive session with P . For that, V generates a unique random prime number for each and every vertex in G (Vertex-Random prime set $R1$) and sends it to P .

4) P satisfies the constraint C for $R1$, X and V^C and encodes satisfied C to a string $\pi1$ and sends it to V .

5) V takes a commitment to the whole $\pi1$ and checks some randomly selected locations in $\pi1$ using PCPs. If $\pi1$ passes all checks done by V , V generates another Vertex-Random prime set $R2$ and sends it to P to start second iteration. Otherwise, V rejects the output V^C with probability 1.

6) Step 4 is repeated for $R2$.

7) Verification process of $\pi2$ (encoded satisfied constraint for $R2$) is same as in step 5. If $\pi2$ passes all checks done by V , V accepts V^C with probability $1-\epsilon$ where ϵ can be made arbitrarily small. Otherwise, V rejects the output with probability 1. Our experimental results have shown that ϵ can be made very small.

Vemaque satisfies the soundness and completeness properties since it uses mechanisms of PPSs: PCPs to check the validity of $\pi1$ and $\pi2$.

Completeness: If output is correct, *Vemaque* allows P to convince the V with probability 1.

Soundness: Otherwise, P can convince V with very low probability ϵ . In *Vemaque*, we have experimentally shown that $\epsilon < 0.01852$ for both KCLQ and MCLQ.

B. Common Work

First, we present the work in *Vemaque* that are common for the verification of both KCLQ and MCLQ.

1) Vertex-Random Prime Set

The generation of Vertex-Random prime set is same for both KCLQ and MCLQ.

Suppose the graph $G = (V, E)$ be the input to the third party computation. Client generates a set of distinct random prime numbers where number of random primes equals to $|V|$. Then a unique random prime is assigned to each and every vertex to produce Vertex-Random prime set R .

Let $V = \{v_1, v_2, v_3, v_4, v_5, v_6\}$ and $E = \{(v_1, v_2), (v_1, v_3), (v_1, v_4), (v_2, v_3), (v_2, v_4), (v_3, v_4), (v_3, v_5), (v_4, v_6), (v_5, v_6)\}$ (See Figure 2). We consider G for the examples in rest of the paper.

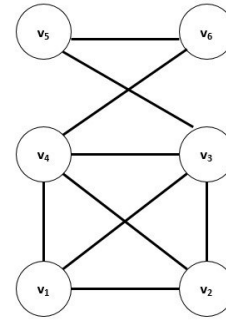


Figure 2 : Graph G

Let $R = \{3, 7, 11, 19, 23, 29\}$ and a Vertex-Random prime set can be generated as follows.

$$v_1 \Rightarrow 3, v_2 \Rightarrow 7, v_3 \Rightarrow 11, v_4 \Rightarrow 19, v_5 \Rightarrow 23, v_6 \Rightarrow 29$$

2) Edge Value

Vemaque introduces a value to each edge in G called as "Edge Value". Suppose, $(v_1, v_2) \in E$ and r_1 and r_2 are the random primes assigned for v_1 and v_2 accordingly. Then, Edge Value of (v_1, v_2) represented as $(v_1, v_2)_{val}$ is calculated by expression (1).

$$(v_1, v_2)_{val} = r_1 + r_2 \quad (1)$$

The set of Edge Values for G can be calculated as, $(v_1, v_2)_{val} \rightarrow 3+7 = 10$, $(v_1, v_3)_{val} \rightarrow 3+11 = 14$,.....

3) Edge Sum

Suppose, $(v_1, v_2), (v_1, v_3), \dots \in E$ and $(v_1, v_2)_{val}, (v_1, v_3)_{val}, \dots$ are their corresponding Edge Values. Edge Sum of E represented as E_{Sum} is calculated by expression (2).

$$E_{Sum} = (v_1, v_2)_{val} + (v_1, v_3)_{val} + \dots \quad (2)$$

E_{Sum} for G can be calculated as, $E_{Sum} = 10 + 14 + 22 + 18 + 26 + 30 + 34 + 48 + 52 = 254$

For a given Vertex-Random number set, there can be two or more different sets of edges ($E_1, E_2, \dots$) that produce the same Edge Sum ($E_{1Sum} = E_{2Sum} \dots$). We have experimentally shown that, the effect of this phenomena can be mitigated by using prime numbers instead of integers in section V.

4) Edge Sum of a complete graph/sub graph (clique)

Suppose, $G^C = (V^C, E^C)$ is a complete graph/sub graph where $|V^C|$ is k . $r_1, r_2, r_3, \dots, r_k$ are the distinct random prime numbers assigned for each vertex in V^C . Since G^C is a complete graph, each vertex in V^C is connected with other $(k-1)$ vertices. Thus, one vertex in V^C appears in $(k-1)$ edges. So, Edge Sum of E^C can be calculated by (3) in *Vemaque* instead of adding Edge Values of E^C one by one.

$$\begin{aligned} E_{Sum}^C &= r_1 * (k-1) + r_2 * (k-1) + \dots + r_k * (k-1) \\ E_{Sum}^C &= (r_1 + r_2 + r_3 + \dots + r_k) * (k-1) \end{aligned} \quad (3)$$

5) Number of edges in complete graph/sub graph

Suppose, $G^C = (V^C, E^C)$ is a complete graph/sub graph. Since G^C is a complete graph, each vertex in V^C is connected with other $k-1$ vertices in V^C . Thus, the number of edges that appear is $(k-1)*k$. It is divided by two since an edge is connected to two vertices. So, $|E^C|$ can be found by expression (4).

$$|E^C| = (k-1) * k/2 \quad (4)$$

C. Verifiable K-Clique Computation

Under this section, we present the arithmetic constraint for KCLQ (C') and how prover and verifier involve for the verification.

1) Arithmetic Constraint C'

Suppose $V^C \subseteq V$ is a clique of size k , found for a given undirected graph $G = (V, E)$ and $G^C = (V^C, E^C)$ is a complete sub graph in G where V^C is the set of vertices. *Vemaque* introduces the arithmetic constraint C' for KCLQ as follows.

$$E_{Sum}^C + (E - E^C)_{Sum} = E_{Sum} \quad (C')$$

In C' , E_{Sum} is the Edge Sum of E and E_{Sum}^C is the Edge Sum of E^C . $E - E^C$ is the set difference of set E and E^C and $(E - E^C)_{Sum}$ is calculated by adding all the Edge Values in $E - E^C$.

E_{Sum}^C covers all the edges assuming G^C is a complete sub graph in G and E_{Sum} covers all the edges in G . $(E - E^C)_{Sum}$ covers all the edges that are not in G^C . So, the union of edges that are covered in E_{Sum}^C and $(E - E^C)_{Sum}$ should equal to edges in G that is covered by E_{Sum} to obtain the consistency of graph.

C' is a sufficient condition to be satisfied if the output V^C is a clique of size k . Satisfying C' doesn't mean that V^C is a clique. But, if C' is not satisfied, then V^C is not definitely a clique. Consider case1.

a) Case 1 : $V^C = \{v_1, v_2, v_3, v_4\}$ and $|V^C| = k=4$ (figure 3)

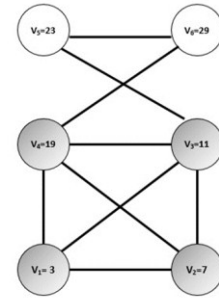


Figure 3 : Case 1 of KCLQ

$$E_{Sum}^C = (3+7+11+19) * (3) = 120 \text{ from (3)}$$

Since V^C is a clique, $E^C = \{(v_1, v_2), (v_1, v_3), (v_1, v_4), (v_2, v_3), (v_2, v_4), (v_3, v_4)\}$. So, $E - E^C = \{(v_3, v_5), (v_4, v_6), (v_5, v_6)\}$

Assume V^C is a clique. Thus it could satisfy C' .

$$E_{Sum}^C + (E - E^C)_{Sum} = E_{Sum}$$

$$120 + \{(v_3, v_5), (v_4, v_6), (v_5, v_6)\}_{Sum} = 254$$

$$120 + \{(v_3, v_5)_{Val} + (v_4, v_6)_{Val} + (v_5, v_6)_{Val}\} = 254$$

$$120 + \{(11+23) + (19+29) + (23+29)\} = 254$$

Since $V^C = \{v_1, v_2, v_3, v_4\}$ is actually a clique, C' is satisfied. And also we can show when V^C is not a clique (e.g. $V^C = \{v_1, v_2, v_3, v_4, v_5\}$) C' is not satisfied since C' is a sufficient condition to be satisfied if the output V^C is a clique of size k .

2) Verification Process

Vemaque allows client verify the computation $F(KCLQ)$ for the input $X(G \text{ and } k)$. Output of F is V^C . The arithmetic constraint C' is satisfied if V^C is a clique. Our experimental results have shown that probability of satisfying C' without the correct answer is lower than the 0.01819. So we can claim that C' is approximately equivalent to the computation F . Server executes F on X and satisfies C' for a received Vertex-Random prime set R with X and V^C .

D. Verifiable Maximum Clique Problem

We introduce some notations before introducing C'' and verification process for MCLQ.

1) Complement of G and Complete Graph of G

Graph $G' = (V, E')$ is the complement of graph $G = (V, E)$ such that, for any edge in G' is not an edge in G . Graph $G'' = (V, E'')$ is the complete graph for the graph $G = (V, E)$ where any two vertices in V is connected by an edge in E'' (Figure 4).

2) Vertex Factor Sum (VFS) and Edge Factor Sum (EFS)

Suppose $V^C \subseteq V$ is the maximum clique found for a given undirected graph $G = (V, E)$ and $G' = (V, E')$ is its complement.

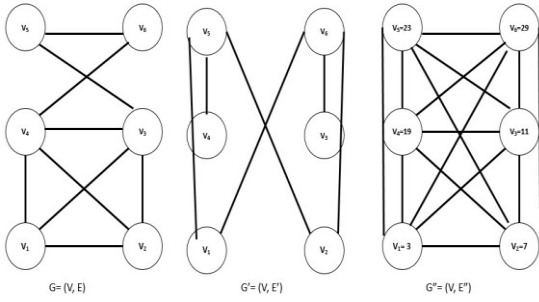


Figure 4 : G and G' and G''

Consider the set $V-V^C$. Select a vertex v from $V-V^C$. Navigate through edges in E' to find all edges that includes v . If such found, remove that edge from E' and take the Edge Value of that edge. Finally, add the Edge Values of all these edges (in which v is an incident). We use the term 'Vertex Factor Sum (VFS)' of v for this addition and "factorizing E' by v " for the process.

Suppose v_1 belongs to $V-V^C$ and $E' = \{(v_1, v_2), (v_1, v_3), (v_2, v_3), \dots\}$. Then the edges factorized by v_1 for E' is $v_1, (v_2, v_3, \dots)$. E' is $\{(v_2, v_3), \dots\}$ after removing $(v_1, v_2), (v_1, v_3), \dots$. We use the term "factors of v_1 for $v_2, v_3, \dots$ ". Among these factors there should be at least one factor (vertex) that doesn't belong to $V-V^C$ to treat all of them as valid factors. Otherwise all factors are treated as invalid in Vemaque.

Suppose, $v_2, v_3, \dots, v_{m+1}$ are the m valid factors of v_1 and $r_1, r_2, \dots, r_{m+1}$ the random prime numbers assigned for each vertex $v_1, v_2, v_3, \dots, v_{m+1}$ accordingly. Then Vemaque finds VFS represented as $v_1(\text{facVal})$ for $v_1 \in V-V^C$ by (5).

$$(v_1)_{\text{facVal}} = m \cdot r_1 + (r_2, r_3, \dots, r_{m+1}) \quad (5)$$

Vemaque finds VFS for all vertices in $V-V^C$ with updated E' . Then, these VFSs are added and we use the term 'Edge Factor Sum (EFS)' for this addition. Edge factor sum of E' by $V-V^C$ is represented as E'_{FacSum} is calculated by (6) where $V-V^C = \{v_1, v_2, v_3, \dots, v_j\}$.

$$E'_{\text{FacSum}} = (v_1)_{\text{facVal}} + (v_2)_{\text{facVal}} + (v_3)_{\text{facVal}} + \dots \quad (6)$$

3) Arithmetic Constraint C''

Suppose $V^C \subseteq V$ is the maximum clique for an undirected graph $G = (V, E)$ and Graph $G' = (V, E')$ is the complement of graph G and $G'' = (V, E'')$ is the complete graph of G . Vemaque introduces the arithmetic constraint C'' for MCLQ.

$$E_{\text{Sum}} + E'_{\text{FacSum}} = E''_{\text{sum}} \quad (C'')$$

In C'' , E_{Sum} is the Edge Sum G and E''_{sum} is the Edge Sum of G'' . E'_{FacSum} is the EFS of E' by $V-V^C$ consisting of valid factors.

It is proven that if maximum clique is V^C for graph G , minimum vertex cover is $V-V^C$ for the complement of that graph G' [24, 26]. So, $V-V^C$ should cover all the edges in G' (E'). E'_{FacSum} covers all edges in E' with minimum number of vertices ($V-V^C$) since, for any vertex $v \in V-V^C$, VFS for that v , consists of valid factors in which there is at least one factor that doesn't belong to $V-V^C$. $(E''_{\text{sum}} - E_{\text{Sum}})$ gives the summation of Edge Values in E' . So $(E''_{\text{sum}} - E_{\text{Sum}})$ should equal to E'_{FacSum} .

C'' is a sufficient condition to be satisfied if the output V^C is maximum clique and satisfying C'' doesn't mean that

V^C is the maximum clique. But, if C'' is not satisfied, then V^C is not definitely the maximum clique. Consider case 1 and case 2.

a) case 1: $V^C = \{v_1, v_2, v_3, v_4\}$ and $V-V^C = \{v_5, v_6\}$ (see Figure 5).

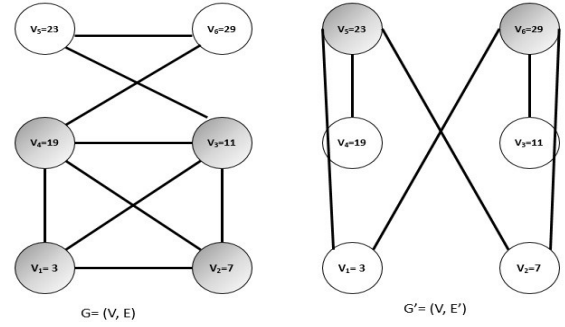


Figure 5 : Case 1 of MCLQ

$E' = \{(v_1, v_5), (v_1, v_6), (v_2, v_5), (v_2, v_6), (v_3, v_5), (v_3, v_6), (v_4, v_5), (v_4, v_6)\}$. Factorize E' by v_5 first: $v_5(v_1, v_2, v_4)$. Factors are valid since none of v_1, v_2 or v_4 belongs to $V-V^C$. Then $(v_5)_{\text{facVal}}$ is calculated. $(v_5)_{\text{facVal}} = 3 \cdot 23 + (3+7+19) = 98$ by (5).

E' is updated to $\{(v_1, v_6), (v_2, v_6), (v_3, v_6)\}$. Factorizing E' by v_6 : $v_6(v_1, v_2, v_3)$. Factors are valid since none of v_1, v_2 or v_3 belongs to $V-V^C$. $(v_6)_{\text{facVal}} = 3 \cdot 29 + (3+7+11) = 108$ (expression (5)). $E' = \{\}$ after factorizing by all vertices in $V-V^C$.

$$E'_{\text{FacSum}} = 98 + 108 = 206 \quad \text{by (6)}$$

$$E''_{\text{sum}} = (3+7+11+19+23+29) \cdot (6-1) = 460 \quad \text{by (3)}$$

Suppose, V^C is maximum clique. Thus, it can satisfy C'' .

$$\begin{aligned} E_{\text{Sum}} + E'_{\text{FacSum}} &= E''_{\text{sum}} \\ 254 + 206 &= 460 \end{aligned}$$

$V^C = \{v_1, v_2, v_3, v_4\}$ is actually the maximum clique. So, C' is satisfied.

b) Case 2 : $V^C = \{v_1, v_2, v_4\}$ and $V-V^C = \{v_3, v_5, v_6\}$. V^C is a clique, but not the maximum clique (Figure 6).

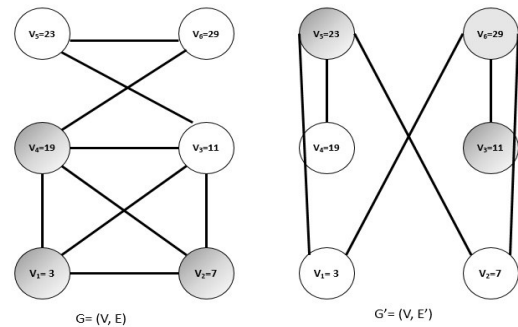


Figure 6 : Case 2 of MCLQ

Factorizing $E' = \{(v_1, v_5), (v_1, v_6), (v_2, v_5), (v_2, v_6), (v_3, v_5), (v_3, v_6), (v_4, v_5), (v_4, v_6)\}$ by v_3 first: $v_3(v_5, v_6)$. Factors are not valid since v_3 doesn't exist at least one factor that doesn't belong to $V-V^C$. Client checks whether the factors are valid or invalid in verification process probabilistically.

4) Verification Process

Vemaque allows client verify the computation F (MCLQ) for the input X (G). Output of F is V^C . The arithmetic constraint C'' is satisfied if V^C is the maximum clique. Our experimental results have shown that probability of satisfying C'' without the correct answer is lower than the 0.01852. So we can claim that C'' is approximately equivalent to F . Server executes F on X and satisfies C'' for a received Vertex-Random prime set R with X and V^C .

V. SIMULATION AND EVALUATION

A. Set up for the Simulation

All simulations are run on the same hardware with the prover on one machine and the verifier on another. V is an Intel dual-Core 2.5 GHz machine with 4 GB memory and Operating system (OS) is Ubuntu 14.04 and P is an Intel corei3 2.2 GHz machine with 8 GB memory and OS is Ubuntu 14.04.

B. Evaluation of K -clique Computation

We have used 34 graphs from the well-known DIMACS benchmark set [21] and 24 random graphs generated by a sequential algorithm [22] for our experiment. The results are given in the table 1.

We tested 58 graphs. *Vemaque* crashed for $\langle G=C2000.10, k=7 \rangle$. Among other 57, server was unable to solve KCLQ for $\langle G=DSJC500_5, k=34 \rangle$ and $\langle G=brock800_4, k=33 \rangle$. So, we consider 55 graphs to measure the accuracy.

Vemaque has verified the result of $\langle G=gen400_p0.9_65, k=59 \rangle$ incorrectly. So, the accuracy of verification for KCLQ is 0.98181(54 / 55) in *Vemaque*. That means, *Vemaque* allows

Table 1 : Experimental result of KCLQ

Instance(G)	k	Solve d	Node s	Edges	Time(mills)	Verif y
Random_1	4	yes	15	63	43	yes
Random_2	6	yes	15	63	123	yes
Random_3	7	yes	15	63	65	yes
Random_4	30	yes	31	473	84	yes
Random_5	16	yes	34	471	300	yes
Random_6	9	yes	38	501	344	yes
Random_7	25	yes	51	612	107	yes
Random_8	34	yes	51	612	140	yes
Random_9	19	yes	60	701	383	yes
Random_10	23	yes	67	754	388	yes
Random_11	29	yes	52	1228	615	yes
Random_12	37	yes	55	1301	518	yes
Random_13	50	yes	61	1402	423	yes
Random_14	55	yes	61	1402	533	yes
Random_15	33	yes	68	1615	833	yes
Random_16	47	yes	73	1708	844	yes
Random_17	51	yes	77	1912	943	yes
Random_18	50	yes	116	1347	760	yes
Random_19	71	yes	116	1347	688	yes
Random_20	10	yes	102	4970	664	yes
Random_21	7	yes	503	28360	1182	yes
Random_22	57	yes	600	30228	1097	yes
Random_23	34	yes	810	41195	1321	yes
Random_24	10	yes	1000	50623	1754	yes
C125.9	32	yes	125	6963	749	yes
C250.9	41	yes	250	27984	1558	yes
C250.9	51	yes	250	27984	1763	yes
C500.9	51	yes	500	112332	3874	yes
C1000.9	68	yes	1000	450079	18365	yes
C2000.10	7	-	2000	999836	-	-
DSJC500_5	7	yes	500	115248	3459	yes

DSJC500_5	34	no	500	115248	1512	-
brock200_2	11	yes	200	9876	1165	yes
brock200_4	8	yes	200	13089	1596	yes
brock400_2	28	yes	400	59786	1703	yes
brock400_4	17	yes	400	59765	2317	yes
brock800_2	21	yes	800	208166	5312	yes
brock800_4	19	yes	800	207643	4329	yes
brock800_4	33	no	800	207643	1156	-
gen200_p0.9_44	41	yes	200	17910	1234	yes
gen200_p0.9_55	55	yes	200	17910	1621	yes
gen400_p0.9_55	55	yes	400	71820	2904	yes
gen400_p0.9_65	63	yes	400	71820	2767	yes
gen400_p0.9_65	59	yes	400	71820	3367	no
gen400_p0.9_75	74	yes	400	71820	2532	yes
hamming10-4	37	yes	1024	434176	13911	yes
hamming8-4	11	yes	256	20864	1501	yes
keller4	8	yes	171	9435	885	yes
keller5	23	yes	776	225990	3190	yes
p_hat300-1	5	yes	300	10933	2245	yes
p_hat300-2	6	yes	300	21928	1567	yes
p_hat300-3	29	yes	300	33390	1730	yes
p_hat700-1	10	yes	700	60999	3308	yes
p_hat700-2	37	yes	700	121728	3065	yes
p_hat700-3	62	yes	700	183010	4000	yes
p_hat1500-1	11	yes	1500	284923	5615	yes
p_hat1500-2	35	yes	1500	568960	7760	yes

client to accept the wrong answers with 0.01819(1-0.98181) for KCLQ. We further investigated the case of $G=brock800_4$ and noticed that C' is satisfied for a correct k -clique that exists in G . But P has sent a wrong output to V and V was unable to correctly verify it with PCP. So, we don't see any satisfaction of C' with incorrect output in our simulation and we can show that the accuracy of C' is 1 for a given set of input and output of KCLQ both theoretically (Section IV) and experimentally. We also computed the accuracy after first iteration and it is too 0.98181(54/55). But MCLQ shows that more iterations provides a higher accuracy.

We also did our experiment for the same set of graph instances mentioned in Table 1 with random integers for two iterations. After the experiment, we found its accuracy as 0.9464 (53/56). In this Simulation, Prover had work with a different set of edges that produces the same edge sum for Random_22 and Verifier was unable to verify it correctly. But In random prime number simulation, Prover had always worked on the set of edges that is identical to the input graph. So the accuracy with random prime numbers is higher.

1) Set up Cost of Verifier

Since we don't make any assumption or bound on computational and memory power of server, we only be concerned about client's set up cost. That is the verification cost of the client. The verification process of client mainly consists of three polynomial time computations: producing Vertex-Random prime set, computing E_{sum} and probabilistic checking. Time complexity of verification in *Vemaque* is $O(|V+E|)$ for KCLQ.

Typically, the number of edges is greater than the number of vertices. So, we further explored the change of verification time against the number of edges as depicted in the graph of Figure 7. This graph was drawn from the

experimental results in Table 1. According to this graph, we can clearly see, when the number of edges increases, the verification time also increases.

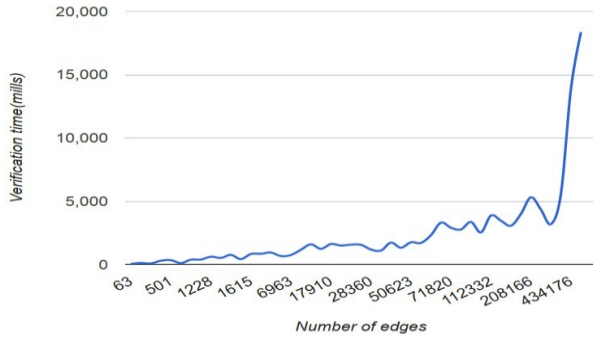


Figure 7 : Verification time against the number of edges

2) Powers and Limitations

We tested our mechanism for graphs with more than 1000 vertices and 250,000 edges. *Vemaque* could provide the verification in less than 30s (Table 1) for those graphs. This is a reasonable verification cost for a NP-complete problem like KCLQ under the machine configurations we used for the client machine.

We also found some limitations of *Vemaque*. When the size of graph increases in terms of number of edges, *Vemaque* stops execution after some graph size, mainly due to hardware limitations. In our set up, *Vemaque* worked for a graph with 2,134,530 edges and 5000 nodes without crashing.

C. Evaluation of maximum clique computation

We have used 28 graphs from the well-known DIMACS benchmark set [21] and 27 random graphs generated by a sequential algorithm [22] for our experiment. The results are given in the table 2. We tested 55 graphs. *Vemaque* terminated when verifying p_hat1500-3. So, we consider only 54 instances for the evaluation process.

Table 2 : Experimental result of MCLQ

Instance(G)	Max_Clique	Nodes	Edges	Time (mills)	Verify
random_0	2	2	1	34	yes
random_1	6	15	63	170	yes
random_2	6	31	49	196	yes
random_3	10	40	112	215	yes
random_4	13	25	160	223	yes
random_5	21	33	273	227	yes
random_6	25	36	372	254	yes
random_7	30	50	435	81	no
random_8	30	50	455	261	yes
random_9	30	50	567	409	yes
random_10	33	53	612	345	yes
random_11	37	64	762	355	yes
random_12	50	51	1227	765	yes
random_13	50	60	1325	357	yes
random_14	52	63	1412	368	yes
random_15	55	100	1315	501	yes
random_16	60	73	1815	812	yes
random_17	67	80	2318	763	yes
random_18	50	116	1429	629	yes
random_19	73	84	2803	811	yes
random_20	100	102	4970	1287	yes
random_21	100	150	5012	2936	yes
random_22	121	400	26512	1798	yes
random_23	110	503	28360	1823	yes
random_24	110	600	30228	2244	yes
random_25	101	810	41195	3793	yes
random_26	101	1000	50623	6127	yes

c125.9	34	125	6963	1168	yes
c250.9	44	250	27984	1734	yes
c500.9	67	500	112332	10201	yes
c1000.9	68	1000	450079	34400	yes
dsjc500_5	13	500	115248	8120	yes
brock200_2	12	200	9876	976	yes
brock200_4	17	200	13089	1709	yes
brock400_2	29	400	59786	4079	yes
brock400_4	33	400	59765	10213	yes
brock800_2	24	800	208166	13067	yes
brock800_4	26	800	207643	13115	yes
gen200_p0.9_44	44	200	17910	1213	yes
gen200_p0.9_55	55	200	17910	1103	yes
gen400_p0.9_55	55	400	71820	7650	yes
gen400_p0.9_65	65	400	71820	8653	yes
gen400_p0.9_75	75	400	71820	7612	yes
hamming10-4	40	1024	434176	27823	yes
hamming8-4	16	256	20864	1635	yes
keller4	11	171	9435	1280	yes
keller5	27	776	225990	15560	yes
p_hat300-1	8	300	10933	1611	yes
p_hat300-2	25	300	21928	1583	yes
p_hat300-3	36	300	33390	2340	yes
p_hat700-1	11	700	60999	4918	yes
p_hat700-2	44	700	121728	10040	yes
p_hat700-3	62	700	183010	11448	yes
p_hat1500-3	-	1500	847244	-	yes
p_hat1500-1	12	1500	284923	16524	yes

Vemaque has verified the result of G=Random_7 incorrectly. So, the accuracy of approximate verification is 0.98148(53/54). Thus, *Vemaque* allows client accept the wrong answers with 0.01852(1-0.98245) for MCLQ. In case G=Random_7, Although C" is not satisfied with the incorrect output, PCP has verified C" as correct. So, We can also show that the accuracy of C" is 1 for a given set of correct input and output of MCLQ both theoretically (Section IV-D) and experimentally. We also considered the accuracy after first iteration. It is 0.96296 (52/54) and this shows the benefit of using 2 iterations.

We did our experiment for the same set of graph instances mentioned in Table 2 with random integers for two iterations. *Vemaque* achieved 0.9636 (53/55) of accuracy for it. In this simulation, Prover worked with different set of edges that produces the same edge sum for graphs, random_2 and keller4. *Vemaque* could verify random_2 correctly, but keller4 incorrectly. Since random prime number simulation didn't produce this phenomena, accuracy is higher with random prime numbers.

1) Set up Cost of Verifier

The verification process of client mainly consists of three polynomial time computations: producing the set of vertex-random prime set, computing Edge E_{Sum} and probabilistic checking. Time complexity of *Vemaque* is $O(V^2)$ for MCLQ. So, we further explored the change of verification time against the number of vertices as depicted in the graph of Figure 8 that was drawn from results in Table 2.

2) Powers and Limitations

We tested our mechanism for graphs with more than 1000 vertices. *Vemaque* could provide the verification in less than 50s as depicted in table 2 for those graphs. So, we think this is a reasonable performance for MCLQ under the machine configurations we used for the client machine.

In our set up, *Vemaque* worked for a graph with 1,200,046 edges and 9112 nodes without crashing in MCLQ.

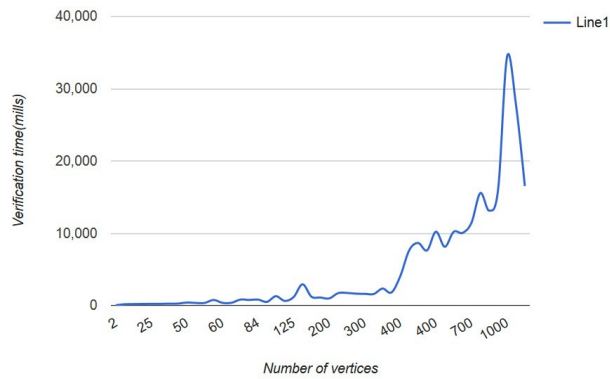


Figure 8 : Verification time against the number of vertices

VI. CONCLUSION AND FUTURE WORKS

Our work extends the concept of verifiable remote computation towards verifying two NP-complete computations: the k-clique and the maximum clique problems that are not handled in prior work, approximately in to a built system: *Vemaque*.

Vemaque can offer a improved accuracy for the verification of both k-clique and maximum clique problem with an accuracy of 0.98181 for k-clique computation and 0.98148 for maximum clique computation. *Vemaque* uses the mechanisms of probabilistic proof systems to achieve a reasonable client performance for a range of graph sizes (works better for graphs with around 10,000 nodes and 2,000,000 edges). When the size of graph increases, the performance (client's verification cost) degrades and *Vemaque* doesn't work after some point of the graph size depending on the configuration of client machine.

The number of random prime numbers generated in *Vemaque* for each iteration is the same as the number of vertices. A 100,000 distinct prime numbers uses a large amount of memory. So the verification process may be slow or sometimes may be impossible when number of vertices is extremely higher. A number theoretically optimal vertex assignment might also be a solution to this problem. But we have proven that, *Vemaque* doesn't depend on either k in KCLQ or size of maximum clique in MCLQ.

In conclusion, there is a great deal of work remaining to bring verifiable computation into practice in terms of reducing client's verification cost, expanding the class of computation that can be handled in the verification and etc. *Vemaque* is a significant step towards that goal.

There are many directions for future work considering many aspects. One is to extend *Vemaque* towards other NP-complete computations. Although we have ignored the server's cost, it is high in *Vemaque* since the server has to produce two approximate certificates. So it is also required to reduce the server's overhead further to bring *Vemaque* in to practical use. And also, it needs to work associating number theories with clique problem to make the mapping theoretically proved.

REFERENCES

- [1] M. Walfish and A. J. Blumberg, "Verifying computations without reexecuting them: from theoretical possibility to near practicality," Electronic Colloquium on Computational Complexity (ECCC), Tech. Rep. TR13-165, Nov. 2013.
- [2] D. P. Anderson et al., "SETI@home: An experiment in public-resource computing," in *Communications of the ACM*, Nov. 2002, Vol. 45.
- [3] P. Mahajan et al., "Cloud storage with minimal trust," in *ACM Trans. on Comp. Sys.*, Dec. 2011, 29(4).
- [4] L. Babai et al., "Checking computations in polylogarithmic time," in *STOC*, 1991.
- [5] O. Goldreichy. (1995). *Probabilistic Proof Systems* [Online]. Available: <http://www.eccc.hpi-web.de/resources/pdf/pps3.pdf>
- [6] Florian Zuleger.(Aug. 2006). *Interactive Proof Systems* [Online]. Available:http://www14.in.tum.de/konferenzen/Jass06/courses/1/Zuleger_Paper.pdf
- [7] B. Parnoet et al., "Pinocchio: Nearly practical verifiable computation," in *IEEE Symposium on Security and Privacy*, pages 238–252, May 2013.
- [8] S. Setty et al., "Resolving the conflict between generality and plausibility in verified computation," in *Proc. EuroSys*, Apr. 2013.
- [9] V. Vuet et al., "A hybrid architecture for interactive verifiable computation," in *IEEE Symposium on Security and Privacy*, 2013.
- [10] S. Goldwasseral et al., "Delegating computation: Interactive proofs for muggles," in *STOC*, 2008
- [11] G. Cormode et al., "Verifying computations with streaming interactive proofs," in *Proc. VLDB Endowment*, 5(1):25–36, 2011.
- [12] J. Thaler et al., "Verifiable computation with massively parallel interactive proofs," in *USENIX HotCloud Workshop*, 2012
- [13] G. Cormode et al., "Practical verified computation with streaming interactive proofs," in *Innovations in Theoretical Computer Science (ITCS '12)*, 2012
- [14] B. Braun et al., "Verifying computations with state," in *SOSP*, 2013.
- [15] S. Setty et al., "Toward practical and unconditional verification of remote computations," in *Workshop on Hot Topics in Operating Systems (HotOS)*, May 2011.
- [16] Vu, V et al., "A hybrid architecture for interactive verifiable computation," in *IEEE Symposium on Security and Privacy*, (May 2013).
- [17] S. Setty et al., "Making argument systems for outsourced computation practical (sometimes)," in *Proceedings of NDSS*, 2012.
- [18] S. Setty et al., "Taking proof-based verified computation a few steps closer to practicality," *USENIX Security*, Aug. 2012.
- [19] J. Winkowski, "Mathematical Foundations of Computer Science," in *Proc. 7th Symposium Zakopane*, Poland, September 1978.
- [20] Y. Ishai et al., "R. Efficient arguments without short PCPs," in *Proc. Computational Complexity (CCC)*, 2007.
- [21] Dimacs. (1992). *Clique Benchmark Instances* [Online]. Available: <https://turing.cs.hbg.psu.edu/txn131/clique.html>
- [22] M. Bayati et al., "A Sequential Algorithm for Generating Random Graphs," in *Algorithmica*, 58(4):860–910, 2010.
- [23] C. Bron and J. Kerbosch J., "Algorithm 457: Finding All Cliques of an Undirected Graph," in *Commun ACM*, 1973, 16(9):575577.
- [24] T. Cormen et al., "NP-Completeness" in *Introduction to Algorithms*, 2nd Edition. MIT Press (McGraw Hill), 2001.
- [25] K.Singh and A. Kumar, "Survey of Algorithms on Maximum Clique Problem," in *Int. Advanced Research J. in Sci., Eng. and Tech.*, Vol. 2, Issue 2, February 2015
- [26] W. Henry et al., "A new approach and faster exact methods for the maximum common sub graph problem," in *11th International Computing and Combinatorics Conference*, Kunming,China, Lecture Notes in Computer Science, volume 3595, pages 717–727. Springer, August,2005